

ISE 305 – Database Systems

Homework 3: Database Design

Spring 2026 – Instructor: Ali Çakmak

Yiğit Aydoğan 150230208
Emir Mete Dönmez 150220205

May 13, 2026

Contents

1	Part I: Entity-Relationship (ER) Modeling	2
1.1	Project Domain	2
1.2	ER Diagram (Chen’s Notation)	2
1.3	Constraint Summary	3
1.4	Advanced Features	3
2	Part III: Relational Schema Translation	3
3	Part II: Relational Schema Translation	3
3.1	Translated Relations	3
3.2	Translation Rationale	4
4	Part III: Functional Dependencies (FDs)	5
4.1	FDs per Relation	5
5	Part IV: Normalization	6
5.1	Initial Schema (HW2 Baseline)	6
5.2	Before-Normalization Functional Dependencies	6
5.3	Step 1: First Normal Form (1NF)	7
5.4	Step 2: Second Normal Form (2NF)	7
5.5	Step 3: Third Normal Form (3NF) & BCNF	7

1 Part I: Entity-Relationship (ER) Modeling

1.1 Project Domain

NutriQuery is a nutritional food database system that consolidates USDA food composition data, brand metadata, health/allergen profiles, and ML-based nutritional scoring predictions. The system supports dietary filtering, range-based nutritional queries, and gap identification for data quality assurance.

1.2 ER Diagram (Chen's Notation)

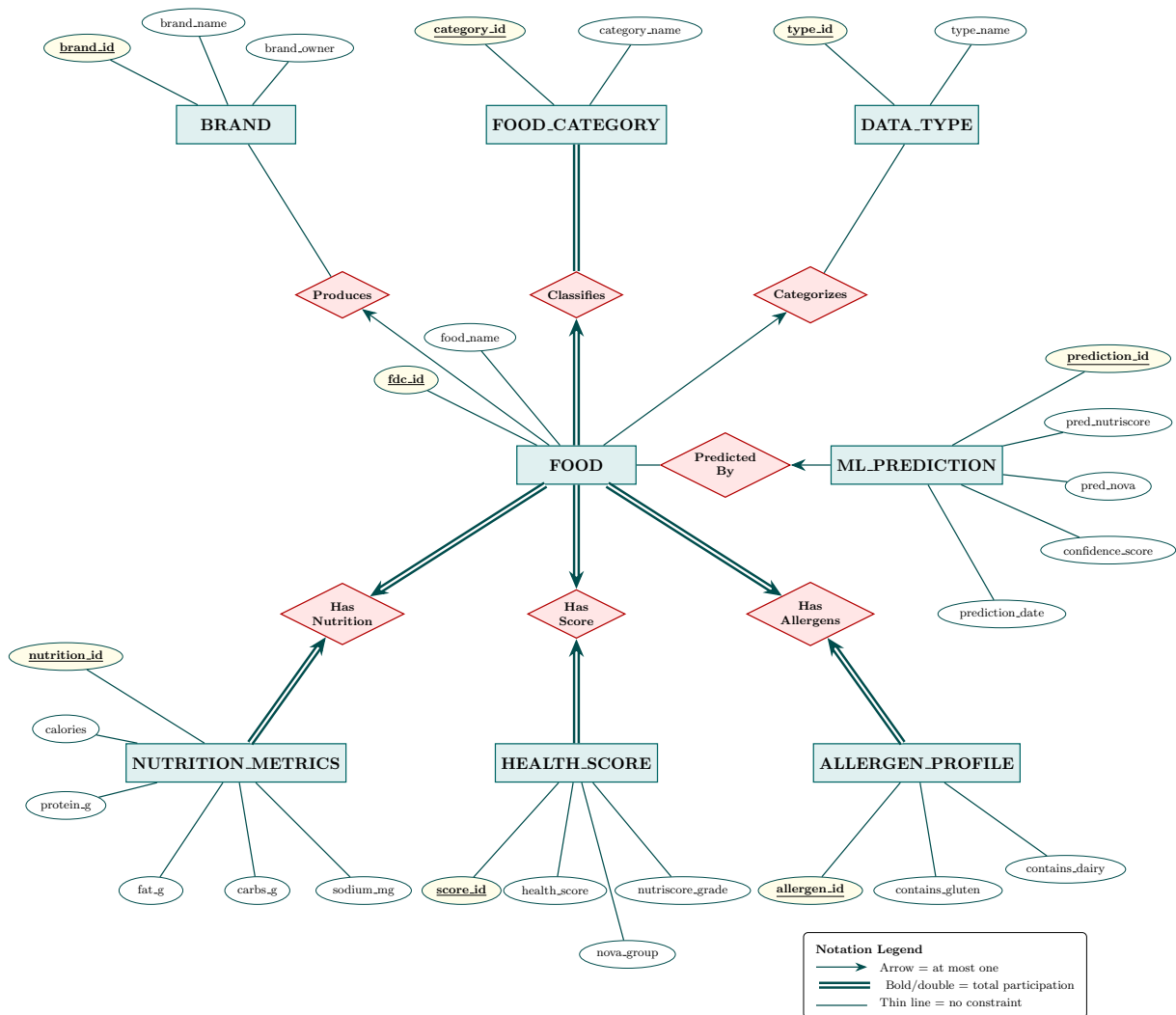


Figure 1: NutriQuery 3NF ER Diagram – Chen's Notation

1.3 Constraint Summary

Relationship	Card.	Left Line	Right Line	Meaning
BRAND – FOOD	1:N	thin	arrow	Partial; each food ≤ 1 brand
FOOD_CATEGORY – FOOD	1:N	double	double+arrow	Total; each food exactly 1 category
DATA_TYPE – FOOD	1:N	thin	arrow	Partial; each food ≤ 1 type
FOOD – NUTR.	1:1	double+arrow	double+arrow	Total both; exactly 1-1
FOOD – HEALTH	1:1	double+arrow	double+arrow	Total both; exactly 1-1
FOOD – ALLERG.	1:1	double+arrow	double+arrow	Total both; exactly 1-1
FOOD – ML-PRED.	1:N	thin	arrow	Partial; each pred. ≤ 1 food

Table 1: Cardinality and participation constraints

1.4 Advanced Features

Our domain does not require weak entities or ISA hierarchies. Every entity has its own strong primary key (either a natural USDA identifier or a surrogate **IDENTITY** key). There are no generalization/specialization hierarchies among food types that would warrant an ISA construct.

2 Part II: Relational Schema Translation

Each strong entity becomes a relation. All relationships are either 1:1 or 1:N, so they are folded into the “many” (or dependent) side via a foreign key. No separate relationship tables are needed (no M:N relationships exist).

2.1 Translated Relations

3NF Relational Schema

```

BRAND (
    brand_id    INT    IDENTITY    PRIMARY KEY,
    brand_name  VARCHAR(255)    NOT NULL,
    brand_owner VARCHAR(255)    NULL
)

FOOD_CATEGORY (
    category_id INT    IDENTITY    PRIMARY KEY,
    category_name VARCHAR(255)    NOT NULL    UNIQUE
)

DATA_TYPE (
    type_id    INT    IDENTITY    PRIMARY KEY,
    type_name  VARCHAR(100)    NOT NULL    UNIQUE
)

FOOD (
    fdc_id      INT                PRIMARY KEY,
    brand_id    INT                FK -> BRAND(brand_id)    NULL,
    category_id INT                FK -> FOOD_CATEGORY(category_id),

```

```

    type_id      INT          FK -> DATA_TYPE(type_id)  NULL,
    food_name    VARCHAR(500) NOT NULL
)

NUTRITION_METRICS (
    nutrition_id INT  IDENTITY PRIMARY KEY,
    fdc_id       INT  NOT NULL  UNIQUE  FK -> FOOD(fdc_id),
    calories     FLOAT NULL,
    protein_g    FLOAT NULL,
    fat_g        FLOAT NULL,
    carbs_g     FLOAT NULL,
    sodium_mg    FLOAT NULL
)

HEALTH_SCORE (
    score_id     INT  IDENTITY PRIMARY KEY,
    fdc_id       INT  NOT NULL  UNIQUE  FK -> FOOD(fdc_id),
    health_score FLOAT NULL,
    nutriscore_grade VARCHAR(50) NULL,
    nova_group   INT  NULL
)

ALLERGEN_PROFILE (
    allergen_id  INT  IDENTITY PRIMARY KEY,
    fdc_id       INT  NOT NULL  UNIQUE  FK -> FOOD(fdc_id),
    contains_gluten BIT  DEFAULT 0,
    contains_dairy BIT  DEFAULT 0
)

ML_PREDICTION (
    prediction_id INT  IDENTITY PRIMARY KEY,
    fdc_id       INT  NOT NULL  FK -> FOOD(fdc_id),
    predicted_nutriscore VARCHAR(50) NULL,
    predicted_nova  INT  NULL,
    confidence_score FLOAT NULL,
    prediction_date DATETIME  DEFAULT GETDATE()
)

```

2.2 Translation Rationale

- **1:N relationships** (Produces, Classifies, Categorizes, Predicted By): The foreign key is placed in the “many” side (FOOD or ML_PREDICTION).
- **1:1 relationships** (Has Nutrition, Has Score, Has Allergens): The foreign key fdc_id with a UNIQUE constraint is placed in the dependent entity, enforcing a strict one-to-one mapping.
- **No M:N tables needed:** All relationships are resolved with embedded foreign keys.
- **No weak entities or ISA hierarchies:** Not applicable to our domain.

3 Part III: Functional Dependencies (FDs)

Below we list all non-trivial functional dependencies for each relation, derived from the real-world semantics of the NutriQuery domain.

Note on Partial and Transitive Dependencies: Because our conceptual ER design translates directly into an optimal schema, the final relations presented here exhibit no partial dependencies (as all primary keys are single-column) and no transitive dependencies. However, to explicitly demonstrate our ability to identify and resolve them as per the assignment requirements, we analyze an unnormalized baseline schema containing transitive dependencies and perform a step-by-step reduction in **Part IV**.

3.1 FDs per Relation

Final 3NF/BCNF Dependencies

BRAND

$$\text{brand_id} \rightarrow \text{brand_name}, \text{brand_owner} \quad (1)$$

FOOD_CATEGORY

$$\text{category_id} \rightarrow \text{category_name} \quad (2)$$

$$\text{category_name} \rightarrow \text{category_id} \quad (\text{UNIQUE constraint} - \text{candidate key}) \quad (3)$$

DATA_TYPE

$$\text{type_id} \rightarrow \text{type_name} \quad (4)$$

$$\text{type_name} \rightarrow \text{type_id} \quad (\text{UNIQUE constraint} - \text{candidate key}) \quad (5)$$

FOOD

$$\text{fdc_id} \rightarrow \text{food_name}, \text{brand_id}, \text{category_id}, \text{type_id} \quad (6)$$

NUTRITION_METRICS

$$\begin{aligned} \text{nutrition_id} \rightarrow \text{fdc_id}, \text{calories}, \text{protein_g}, \text{fat_g}, \\ \text{carbs_g}, \text{sodium_mg} \end{aligned} \quad (7)$$

$$\begin{aligned} \text{fdc_id} \rightarrow \text{nutrition_id}, \text{calories}, \text{protein_g}, \text{fat_g}, \\ \text{carbs_g}, \text{sodium_mg} \end{aligned} \quad (8)$$

HEALTH_SCORE

$$\begin{aligned} \text{score_id} \rightarrow \text{fdc_id}, \text{health_score}, \text{nutriscore_grade}, \\ \text{nova_group} \end{aligned} \quad (9)$$

$$\begin{aligned} \text{fdc_id} \rightarrow \text{score_id}, \text{health_score}, \text{nutriscore_grade}, \\ \text{nova_group} \end{aligned} \quad (10)$$

ALLERGEN_PROFILE

$$\text{allergen_id} \rightarrow \text{fdc_id}, \text{contains_gluten}, \text{contains_dairy} \quad (11)$$

$$\text{fdc_id} \rightarrow \text{allergen_id}, \text{contains_gluten}, \text{contains_dairy} \quad (12)$$

ML_PREDICTION

$$\text{prediction_id} \rightarrow \text{fdc_id}, \text{predicted_nutriscore}, \text{predicted_nova}, \\ \text{confidence_score}, \text{prediction_date} \quad (13)$$

Note: `fdc_id` is *not* a candidate key in `ML_PREDICTION` since one food can have multiple predictions over time.

4 Part IV: Normalization

To mathematically ensure our final schema (presented in Parts I-III) is free from anomalies, we evaluate the database design against functional dependencies using the standard normalization process. We start from an unnormalized baseline (our prior HW2 schema) and systematically derive the 3NF/BCNF relations.

4.1 Initial Schema (HW2 Baseline)

The prior version of our database contained the following 5 tables, which suffered from transitive dependencies and grouped multiple concerns into single relations:

Unnormalized HW2 Schema

```
Brands(brand_id, brand_name, brand_owner, ecoscore_grade)
Foods(fdc_id, brand_id, food_name, data_type, food_category)
Nutrition_Metrics(nutrition_id, fdc_id, calories, protein_g,
                  fat_g, carbs_g, sodium_mg)
Health_and_Allergens(profile_id, fdc_id, contains_gluten,
                    contains_dairy, health_score,
                    nutriscore_grade, nova_group)
ML_Predictions(prediction_id, fdc_id, predicted_nutriscore,
               predicted_nova, confidence_score, prediction_date)
```

4.2 Before-Normalization Functional Dependencies

Based on the unnormalized schema and domain semantics, we identify the following initial functional dependencies (FDs) representing the business rules *before* normalization:

Initial Functional Dependencies

```
FD1: fdc_id  $\rightarrow$  food_name, brand_id, data_type, food_category
FD2: brand_id  $\rightarrow$  brand_name, brand_owner, ecoscore_grade (Anomalous dependency: ecoscore actually depends on the product fdc_id, not the brand).
FD3: nutrition_id  $\rightarrow$  fdc_id, calories, protein_g, fat_g, carbs_g, sodium_mg
FD4: profile_id  $\rightarrow$  fdc_id, contains_gluten, contains_dairy,
      health_score, nutriscore_grade, nova_group
```

FD5: $\text{prediction_id} \rightarrow \text{fdc_id}, \text{predicted_nutriscore}, \text{predicted_nova}, \text{confidence_score}, \text{prediction_date}$

4.3 Step 1: First Normal Form (1NF)

Requirement: All attributes must be atomic (single-valued); no repeating groups.

Analysis: The initial schema already satisfies 1NF because every column stores a single scalar value (INT, FLOAT, VARCHAR, BIT, DATETIME), and there are no multi-valued or array-type attributes.

Result: No changes needed. ✓ 1NF satisfied.

4.4 Step 2: Second Normal Form (2NF)

Requirement: Must be in 1NF, and every non-key attribute must be fully functionally dependent on the *entire* primary key (no partial dependencies).

Analysis: Every table in the original schema uses a **single-column primary key** (brand_id, fdc_id, nutrition_id, profile_id, prediction_id). Partial dependencies can only arise with composite keys.

Result: No changes needed. ✓ 2NF satisfied.

4.5 Step 3: Third Normal Form (3NF) & BCNF

Requirement: Must be in 2NF, and no non-key attribute may transitively depend on the primary key (i.e., $X \rightarrow Y \rightarrow Z$ where Y is not a candidate key). We evaluate the baseline tables against our FDs to eliminate update, insertion, and deletion anomalies.

Analysis – Four major violations found and fixed:

Violation 1: ecoscore_grade in Brands

ecoscore_grade is a product-level environmental score, not a brand-level property. Different products from the same brand have different eco-scores. This violates FD semantics.

Fix: Remove ecoscore_grade from BRAND.

Violation 2: food_category in Foods

food_category is a repeating VARCHAR string. Renaming a category requires updating every matching row $\Rightarrow$ **update anomaly**.

Fix: Extract to a new FOOD_CATEGORY(category_id, category_name) table. Replace the VARCHAR column with an FK category_id in FOOD.

Violation 3: data_type in Foods

Identical issue to food_category – a repeating string like “SR Legacy.

Fix: Extract to DATA_TYPE(type_id, type_name). Replace with FK type_id in FOOD.

Violation 4: Mixed concerns in Health_and_Allergens

This table groups two functionally independent attribute sets: Scoring (`health_score`) and Allergens (`contains_gluten`). Allergen status does not determine health scores. Grouping them violates single-concern design and creates insertion/deletion anomalies.

Fix: Decompose into:

- `HEALTH_SCORE(score_id, fdc_id, health_score, nutriscore_grade, nova_group)`
- `ALLERGEN_PROFILE(allergen_id, fdc_id, contains_gluten, contains_dairy)`

Result: After these extractions, the original schema grows from 5 to 8 tables. Every determinant in the resulting 8 tables is a candidate key, meaning the schema fully satisfies **Boyce-Codd Normal Form (BCNF)**.

This step-by-step standard normalization process mathematically proves the integrity of the final 3NF schema modeled and translated in the preceding sections.